

Cloud, Réseaux Virtuels et Conteneurisation

UE CRV — Sorbonne Université

Naceur.Malouch@lip6.fr

Cours 4 : Orchestration des conteneurs & Kubernetes

Mars 2026

Table des matières

1	Orchestration des conteneurs	2
1.1	Définition	2
1.2	Allocation de ressources	2
1.3	Autoréparation (Self-Healing)	2
1.4	Auto-scaling (remise à l'échelle automatique)	2
1.4.1	À la hausse	2
1.4.2	À la baisse	3
1.5	Exemples de logiciels d'orchestration	3
2	Kubernetes	3
2.1	Architecture globale	3
2.2	Composants du Master node	3
2.3	Pods – Unité de base	4
2.4	Exemple de déploiement YAML	4
3	Réseau virtuel dans Kubernetes	5
3.1	Plugin réseau et CNI	5
3.2	Ce que fait le réseau virtuel	5
3.3	Exemples de plugins réseau CNI	5
4	Récapitulatif	6
5	Sources	6

Orchestration des conteneurs

Définition

Orchestration – Définition

Un logiciel et/ou des outils dont l'objectif est de **simplifier la gestion et le déploiement des conteneurs** et donc des applications conteneurisées.

→ La mise en place des services et l'utilisation des ressources seraient **simplifiées et optimisées**.

Fonctionnalités assurées par l'orchestration :

- Exécuter une application “*quelque part*” (placement automatique)
- Augmenter/diminuer adéquatement le nombre de conteneurs
- Surveillance de l'état des applications et **autoréparation** (*Self-Healing*)
- Simplifier le stockage distribué
- Allocation dynamique et automatique des ressources
- Réplication et **équilibre de charge**
- Gestion de réseaux et notamment des adresses IP

Allocation de ressources

Problème : **Comment ordonnancer les conteneurs parmi les serveurs (hôtes) ?**

Plusieurs stratégies de placement sont possibles :

- Exécuter les nouveaux conteneurs sur le **serveur le moins chargé**
- **Bin-packing** : maximiser la densité de conteneurs par serveur
- Autres critères (localisation, affinité, ressources disponibles...)

Autoréparation (Self-Healing)

Relancer automatiquement les conteneurs défectueux en vérifiant :

- Si le **conteneur s'exécute normalement** (liveness probe)
- Si l'**application du conteneur est prête à répondre** (readiness probe)
- → Généralement avec un **sondage actif** (active probing)

Auto-scaling (remise à l'échelle automatique)

À la hausse

- **Horizontal scaling** : lancement de davantage de conteneurs pour la même application
- **Vertical scaling** : augmentation des ressources (RAM, CPU) pour les conteneurs existants

Déclencheurs typiques :

- Utilisation CPU supérieure à un seuil prédéfini
- Taille des données en mémoire supérieure à un seuil prédéfini
- Augmentation de la charge du trafic réseau

À la baisse

Réduction du nombre de conteneurs ou des ressources allouées lorsque la charge diminue (pour optimiser les coûts).

Exemples de logiciels d'orchestration

Logiciel	Éditeur / Origine
Kubernetes	Google (open source – CNCF)
Docker Swarm	Docker Inc.
NOMAD	HashiCorp
Marathon	Mesosphere (Apache Mesos)

Kubernetes**Architecture globale**

Un nœud Kubernetes peut être une VM ou une machine physique.

Kubernetes est organisé en :

- Un **Master node** (plan de contrôle) qui orchestre l'ensemble
- Un **Cluster** de nœuds workers qui exécutent les conteneurs
- Interaction via **CLI** (kubectl), **API** ou **UI**

Composants du Master node

Composant	Rôle
API server	Point d'entrée pour toute interaction avec le cluster (CLI, UI, API REST)
Controller manager	Surveillance des nœuds et des services, contrôle d'accès, mesures
Scheduler	Placement des pods sur les nœuds workers (algorithme de scheduling)
etcd	Base de données clé-valeur distribuée : stocke tous les paramètres et l'état du cluster

Pods – Unité de base

Pod – Définition

Un pod contient typiquement **un seul conteneur**, ou plusieurs conteneurs si l'application le requiert.

Chaque pod a un namespace réseau différent (une pile réseau isolée).

Le réseau virtuel des pods est basé sur les outils classiques :

- Liens **veth**
- **Bridges** virtuels
- **iptables**
- Etc.

Exemple de déploiement YAML

Fichier de configuration `myserver-deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myserver-deployment
  labels:
    app: myserver
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myserver
  template:
    metadata:
      labels:
        app: myserver
    spec:
      containers:
        - name: myserver
          image: myserver:4.11
          ports:
            - containerPort: 80
```

Déploiement et vérification avec `kubectl` :

```
root@linux# kubectl apply -f myserver-deployment.yaml
Deployment.apps/myserver-deployment created

root@linux# kubectl get deployment
NAME                    READY    UP-TO-DATE    AVAILABLE    AGE
myserver-deployment    2/2      2             2            44s

root@linux# kubectl get pod
NAME                    READY    STATUS    RESTARTS
AGE
```

myserver-deployment-31c72d43-ngc84 61s	1/1	Running	0
myserver-deployment-31c72d43-plqbx 60s	1/1	Running	0

replicas: 2 → Kubernetes crée et maintient automatiquement **2 instances** du pod **myserver**. Si un pod tombe, il est relancé automatiquement (*self-healing*).

Réseau virtuel dans Kubernetes

Plugin réseau et CNI

CNI – Container Network Interface

Interface standardisée qu'un plugin réseau doit implémenter pour être utilisé par Kubernetes. Ce plugin **n'est pas développé par Kubernetes lui-même**.

Responsabilités du CNI :

- **Rajouter** un conteneur au réseau virtuel (ou modifier la configuration)
- **Supprimer** le conteneur du réseau (ou annuler les modifications)
- **Vérifier** que le réseau virtuel fonctionne correctement et renvoyer des erreurs si nécessaire

Ce que fait le réseau virtuel

- Création des **namespaces réseau** pour les pods/conteneurs
- Création des **interfaces réseau** → liens **veth**
- Configuration du **routage**
- Configuration des **bridges Ethernet**
- **Allocation des adresses IP**
- Création des règles de **translation d'adresses (NAT)**
- Etc.

Exemples de plugins réseau CNI

Plugin	Caractéristiques principales
Flannel	Simple, overlay VXLAN, adapté aux petits clusters
Calico	Réseau L3, politiques réseau avancées (NetworkPolicy)
OVN-Kubernetes	Basé sur Open Virtual Network (OVS)
Antrea	Basé sur Open vSwitch, bonnes performances

Il est possible de développer un plugin CNI simple avec les outils Linux classiques : **ip link, ip route, ip netns**

Récapitulatif

Concept	Définition / Rôle
Orchestration	Gestion automatisée du cycle de vie des conteneurs (déploiement, scaling, réparation)
Kubernetes	Orchestrateur open source, architecture Master/Workers, basé sur les pods
Pod	Unité de base Kubernetes = 1 ou quelques conteneurs avec un namespace réseau propre
etcd	Base de données distribuée stockant l'état du cluster
Scheduler	Choisit sur quel nœud placer chaque pod
CNI	Interface standardisée pour les plug-ins réseau de Kubernetes
Self-Healing	Relance automatique des pods défectueux
Auto-scaling	Ajustement dynamique du nombre de conteneurs selon la charge

Sources

- Documentation Kubernetes : <https://kubernetes.io/docs/>
- Container Network Interface (CNI) : <https://github.com/containernetworking/cni>
- Flannel : <https://github.com/flannel-io/flannel>
- Calico : <https://www.tigera.io/project-calico/>
- Copyright Naceur.Malouch@lip6.fr – All rights reserved.